

Templates 2 & 3 Experiments

Thomas Ceresa

June 16, 2026

Closures. LLM scanners plateau after 1–2 iterations, returning the same well-known entities and missing niche ones. I propose the *closures* contribution which fires when a scan stalls (empty or repeated response) and repeats a continuation strategy.

MOTIVATIONAL — appends “*You can do it! Dig deeper, list less well-known entities*” to push beyond the plateau.

TELEPHONE (*fr. téléphone arabe*) — appends “*Your friend is stuck. You must help him to find other names ...*”

COACH — appends “*Your answer is good, but you should look more about this*”, with the rest of the prompt is given by an external “coach” LLM. However this latter did not show conclusive results.

1 Template 2 — Triple Pattern with FILTER

Pattern. SELECT ?x WHERE { ?x p ?y } FILTER(C). **Setup.** GPT-4o-mini (Azure, temp=0), $N = 5$, P/R/F1 mean $\pm$ std. RW1 rewrites FILTER(?x $\in$ S) into seeds $v(?x) = S$ before scanning.

Queries.

- **q1** (low, GT=3): (?country, capital, ?y), ?country $\in$ {France, Bhutan, Eritrea}
- **q2** (high, GT=58): (?film, directed by, ?director), ?director $\in$ {Kubrick, Nolan, Tarantino, Lynch}
- **q3** (medium, GT=17): (?artist, genre, ?y), ?artist $\in$ {Beatles, Radiohead}, ?y $\neq$ pop
- **q4** (medium, GT=38): (?country, area, ?y), 500,000 < ?y < 2,000,000
- **q5** (high, GT=90): (?country, population, ?y), ?y > 10,000,000

Strategies. NL — natural language single call. SPARQL — raw SPARQL fed to LLM. LLMSCAN — RW1 $\rightarrow$ KeyScan/SeedScan/TripleScan $\rightarrow$ LLMConfCond (inject vs post-filter). LLMSCAN_KO — RW1 + force KeyScan. LLMSCAN_PF — no RW1, all conditions in post-filter. LLMSCAN_M / _TEL — LLMScan + motivational / telephone closure (q4–q5 only).

Q	Strategy	GT	P	R	F1	t(s) / tok
q1	NL	3	1.000 $\pm$.000	1.000 $\pm$.000	1.000$\pm$.000	1.0s / 76
	SPARQL		1.000 $\pm$.000	1.000 $\pm$.000	1.000$\pm$.000	0.9s / 102
	LLMScan		1.000 $\pm$.000	1.000 $\pm$.000	1.000$\pm$.000	3.3s / 591
	LLMScan_KO		1.000 $\pm$.000	1.000 $\pm$.000	1.000$\pm$.000	3.0s / 407
	LLMScan_PF		1.000 $\pm$.000	0.333 $\pm$.000	<u>0.500$\pm$.000</u>	5.4s / 2066
q2	NL	58	0.951 $\pm$.003	0.335 $\pm$.020	0.495 $\pm$.022	1.6s / 191
	SPARQL		0.900 $\pm$.056	0.128 $\pm$.015	<u>0.223$\pm$.025</u>	1.1s / 131
	LLMScan		0.958 $\pm$.002	0.393 $\pm$.014	0.557 $\pm$.015	7.5s / 1143
	LLMScan_KO		0.960 $\pm$.002	0.414 $\pm$.024	0.578$\pm$.024	6.6s / 965
	LLMScan_PF		0.000 $\pm$.000	0.000 $\pm$.000	<u>0.000$\pm$.000</u>	4.3s / 1248
q3	NL	17	0.738 $\pm$.053	0.294 $\pm$.000	<u>0.420$\pm$.008</u>	0.9s / 93
	SPARQL		0.611 $\pm$.031	0.294 $\pm$.000	0.397 $\pm$.007	1.0s / 121
	LLMScan		0.871 $\pm$.008	0.400 $\pm$.026	0.548$\pm$.027	4.2s / 687
	LLMScan_KO		0.864 $\pm$.010	0.377 $\pm$.032	0.524 $\pm$.033	3.4s / 518
	LLMScan_PF		0.000 $\pm$.000	0.000 $\pm$.000	<u>0.000$\pm$.000</u>	2.8s / 720
q4	NL	38	0.580 $\pm$.130	0.153 $\pm$.034	<u>0.242$\pm$.054</u>	0.8s / 109
	SPARQL		0.830 $\pm$.142	0.510 $\pm$.040	0.624$\pm$.029	1.1s / 162
	LLMScan		0.267 $\pm$.195	0.300 $\pm$.187	0.281 $\pm$.188	30.8s / 5108
	LLMScan_M		0.283 $\pm$.052	0.616 $\pm$.024	0.386 $\pm$.051	21.0s / <u>18447</u>
	LLMScan_Tel		0.515 $\pm$.045	0.426 $\pm$.086	0.466 $\pm$.068	12.3s / 6624
q5	NL	90	0.619 $\pm$.102	0.911 $\pm$.016	0.734$\pm$.069	4.0s / 538
	SPARQL		0.516 $\pm$.029	0.904 $\pm$.022	0.657 $\pm$.025	3.6s / 637
	LLMScan		1.000 $\pm$.000	0.156 $\pm$.000	<u>0.269$\pm$.000</u>	6.0s / 959
	LLMScan_M		0.505 $\pm$.029	0.720 $\pm$.086	0.593 $\pm$.045	30.3s / <u>24602</u>
	LLMScan_Tel		1.000 $\pm$.000	0.291 $\pm$.012	0.451 $\pm$.015	10.6s / 6524

Table 1: T2 results ($N = 5$). **Bold** = best F1 per query. Underline = worst F1 or worst token cost.

Discussion. RW1 is indispensable. LLMScan_PF scores F1=0 on q2/q3 — without seeds, the scanner generates arbitrary entities all discarded by post-filter. RW1 gives the LLM the constraint set directly, removing the need to reason about membership. *Numeric γ is unstable.* The LLM suffers with numeric thresholds per-entity (std up to 0.203 on q4/q5). SPARQL offloads numeric checking and dominates q4; NL retrieves the most prominent countries

naturally without threshold reasoning, dominating q5. *Closures help on numeric queries but token cost is a concern.* Tel wins on q4 (+0.185 F1, $\times 1.3$ tok): targeted name suggestions prevent hallucination. Motiv triples recall on q5 (0.156 $\rightarrow$ 0.720) by pushing past saturation, but at $\times 25$ tokens and P=0.505 — encouragement without guidance leads to hallucinations.

2 Template 3 — Two-Hop Path Query

Pattern. SELECT ?b WHERE ($s, p_1, ?b$) $\wedge$ ($?b, p_2, t$), s and t fixed. **Setup.** Same as T2.

Strategies. GD: $\text{scan}(s, p_1, ?b) \rightarrow v(?b) \rightarrow \text{scan}(?b, p_2, t)$. DG: reverse direction. JOIN: both independent $\rightarrow \text{path_join}(T_1, T_2)$ on ?b. SIMPLESCAN: single LLM call. MUGALLOIS: LLMSimpleConf $\rightarrow$ SimpleScan if conf $> \tau_S$, else LLMDirectionConf $\rightarrow$ GD/DG/Join + motivational closure on hop 1.

Queries.

- **q1** (low, GT=4, **GD optimal**): (Oscar Wilde, wrote, ?b) $\wedge$ (?b, genre, comedy)
- **q2** (medium, GT=9, **DG optimal**): (United Nations, member, ?b) $\wedge$ (?b, language, Portuguese)
- **q3** (high, GT=21, **Join optimal**): (Africa, country, ?b) $\wedge$ (?b, member of, Commonwealth)

Q	Strategy	GT	P	R	F1	t(s) / tok
q1	NL	4	1.000 $\pm$.000	0.800 $\pm$.112	0.886 $\pm$.064	1.4s / 85
	SimpleScan		1.000 $\pm$.000	0.650 $\pm$.137	0.781 $\pm$.104	1.0s / 113
	GD		1.000 $\pm$.000	0.850 $\pm$.224	0.905 $\pm$.147	5.5s / 1045
	GD_Motiv_closure		1.000 $\pm$.000	1.000 $\pm$.000	1.000$\pm$.000	9.2s / 1630
	DG		0.000 $\pm$.000	0.000 $\pm$.000	<u>0.000$\pm$.000</u>	6.6s / 1008
	MuGalois		0.900 $\pm$.224	0.750 $\pm$.306	0.809 $\pm$.276	9.0s / <u>2975</u>
q2	NL	9	0.875 $\pm$.000	0.778 $\pm$.000	0.824 $\pm$.000	1.8s / 101
	SimpleScan		0.857 $\pm$.000	0.667 $\pm$.000	<u>0.750$\pm$.000</u>	1.0s / 120
	GD		0.857 $\pm$.000	0.667 $\pm$.000	<u>0.750$\pm$.000</u>	10.3s / 3456
	DG		1.000 $\pm$.000	0.889 $\pm$.000	0.941$\pm$.000	6.0s / 907
	MuGalois		0.949 $\pm$.070	0.845 $\pm$.099	0.892 $\pm$.083	11.2s / 1960
q3	NL	21	0.978 $\pm$.030	0.724 $\pm$.078	0.829 $\pm$.039	1.4s / 111
	GD		0.924 $\pm$.013	0.591 $\pm$.093	<u>0.717$\pm$.078</u>	16.5s / 2267
	DG		1.000 $\pm$.000	0.724 $\pm$.052	0.839 $\pm$.034	8.6s / 1672
	Join		1.000 $\pm$.000	0.905 $\pm$.021	0.945$\pm$.012	8.0s / 1474
	MuGalois		1.000 $\pm$.000	0.895 $\pm$.021	0.945$\pm$.012	8.0s / 1714

Table 2: T3 results ($N = 5$). **Bold** = best F1. Underline = worst F1 or worst token cost. MuGalois: q1 $\rightarrow$ GD+closure, q2 $\rightarrow$ DG, q3 $\rightarrow$ Join.

Discussion.

Direction matters critically. Choosing the wrong starting side is catastrophic: DG on q1 yields F1=0 because scanning *comedy genre* without context returns thousands of random works with no Wilde constraint. GD on q2 wastes $14\times$ more tokens than DG — iterating over 193 UN members to filter 9 Portuguese speakers is far less efficient than starting from the 9 speakers directly.

Motivational closure unlocks q1. GD alone stalls after 2–3 comedies — the LLM exhausts its obvious answers. The closure fires on stall and pushes it further, recovering the missing *A Woman of No Importance* with P=1.000 at $\times 1.6$ tokens.

Join is optimal when $|A| \approx |B|$. Africa (54 countries) $\approx$ Commonwealth (56 members) — neither side dominates, so independent scans intersected in memory achieve P=1.000, F1=0.945. LLMDirectionConf detects this balance via A/B cardinality reasoning and correctly falls back to Join.

MuGalois selects correctly on all three queries after fixing DirectionConf, matching the best strategy on q2 and q3. Token overhead from confidence and direction calls is modest but non-negligible — NL achieves comparable F1 at 10–30 $\times$ fewer tokens on well-known domains, suggesting decomposition is mere interesting to use it on niche or complex paths.

Futurs works (in my opinion) : a new template for sure; continue to reduce the number of tokens ; and find a better way to determine the best model before the LLMs’ calls;